Triangle decompositions II: Formalization and reproducibility companion

Juan Pablo Traverso Gianini
Independent researcher, Santiago, Chile
jtraverso@gmail.com
ORCID: 0009-0003-6068-4096

Companion to Paper II: *Triangle decompositions II: Robust absorption for typed triangles* [1], preprint 1.0.

Preprint 1.0 — 20 October 2026.

DOI: https://doi.org/10.5281/zenodo.23502224.

Copyright © 2026 Juan Pablo Traverso Gianini. This document is licensed under CC BY 4.0.

1. Validated scope

Three explicit targets were compiled in the aligned campaign: `PartitionedTriangleAbsorption.PaperII.Core`, `PartitionedTriangleAbsorption.PaperII.Families`, and `PartitionedTriangleAbsorption.PaperII.PaperIApplications`. Their overlapping import closures contain 1,510 project modules. The campaign began with an empty project output directory and compiled every module from source; 623 were compiled in the Paper I phase and 887 in the Paper II phase. The pinned Mathlib cache was reused. The fifteen final axiom queries passed with only `propext`, `Classical.choice` and `Quot.sound`.

The result includes actual existence certificates for the seventeen conserved families. It also includes the exhaustive census, the bridge from representatives to the construction library and transport back to the original labelling. Together these prove the typical-host property for every family of three-element multisets on four labels that uses every label and gives every label a forbidden partner. Neither a Latin producer nor membership in the catalogue is assumed in the final theorem.

The completed local campaign is recorded as `PASS_CLEAN_ALIGNED_BUILD`; it is not an independent reproduction. Revision v2 subsequently corrects author headers only and has status `PASS_HEADER_ONLY_REVISION`; selected revision v4 has status `PASS_README_ONLY_REVISION` and changes only the archive README and its manifest. Every proof-body byte is preserved. Historical receipts describe the sources before that correction, and the reversible verification in Section 5 reconstructs their exact bytes. No new Lean compilation is claimed for the corrected headers.

2. Frozen sources

The selected source archive is `PAPER_II_LEAN_20261020_ALIGNED_v4.zip`, at the immutable source reference. Its SHA-256 is:

`677faa70a76444a000b8c2192a45d0ab0e39f8899825c004fdf6cbe52bb755cc`.

The archive contains 4,562 files and its `SHA256SUMS.txt` checks 4,561 entries. The unchanged campaign report `evidence/verification.json` has SHA-256:

`8b609b174b9f16086c2faf243e2cb828d34c4110f220e59556482610f519ce59`.

The toolchain is exactly `leanprover/lean4:v4.35.0-rc3`. Mathlib is pinned to `c55e6e786f49471c72fbddbec5415808896aec1e`; the lock file and `evidence/environment.json` identify its supporting packages.

The clean predecessor is `PAPER_II_LEAN_20261020_ALIGNED_v1`, SHA-256 `2ed5e683dae4ff9ab4a24187957dbd2fb3ec18d04e7236c88dd4c6c38ac079b7`. Revision v2 adds 1,352 missing author headers, corrects 59 surname spellings and retains 99 already correct headers. All 1,510 project Lean files identify Juan Pablo Traverso Gianini and Apache-2.0. `LICENSE` and `source/LICENSE` supply that licence for original packaging materials and project sources. The nine dependency licence texts in `dependencies/licenses/` are retained unchanged. Revisions v3–v4 correct the README's paper title, UTF-8 typography and scope of historical note references; its other 4,560 files are byte-identical to v2, including all sources, attribution data, licences and evidence.

The earlier Paper II closure contained 1,511 modules. Alignment replaces the import and query for `FourColourConsumerStage84` in `PaperIApplications` with the bridge used by Paper I. The old closure loses `FourColourConsumerStage84` and `FourClassGraphConsumerStage80` and gains `PaperII.FourColourApplication`; all other common source files are identical except the entry file's import and query. Before the author-header revision, the relocated module was byte-for-byte the former `RequestProject.NW18.FourColourPaperII` from the original Paper I freeze. In the selected archive its body remains identical, while its author header is corrected. Its namespace and theorem statements are unchanged. `evidence/ALIGNMENT_MANIFEST.json` records the alignment.

Paper I now selects `PAPER_I_LEAN_20261020_ALIGNED_v3`, with 1,070 modules and six final queries from the same aligned campaign. Its 623 shared modules and this archive use the same proof bodies and corrected headers. Each paper retains its own closure, receipts and entry queries.

3. Correspondence and entry points

Here `PTA` abbreviates `PartitionedTriangleAbsorption` for reading; the sources do not define that alias.

Written result	Declaration
Property (2.7)	`PTA.BlowupB.KeevashTypedSpec`
Theorem A	`PaddedGeneral.RealInverse.keevashTypedSpec_of_threeStep Links`
Theorem B	`PTA.BlowupB.teoremaB_of_keevash`
Balanced deduction	`PTA.BlowupB.teoremaB_balanced`
Corollary 4.2	`PaddedGeneral.RealInverse.nearlyComplete_F123`
Idealized F1/F2	`PaddedGeneral.RealInverse.idealized_F12`
Theorem C	`PTA.BlowupB.F0Direct.theorem146_ordered`
Lemma SC	`PTA.BlowupB.LemmaSC.absorber_onePart`

Written result	Declaration
Physical realization	`PTA.BlowupB.LatinCore.RootPhysicalRefiner.exists_refiner`
Physical producer	`PTA.BlowupB.LatinCore.PhysicalProducer.physicalCycRefiner3`
Seventeen certificates	`PTA.BlowupB.LatinCore.Conserved17.all_family_certificates`
Seventeen conclusions	`PTA.BlowupB.LatinCore.Conserved17.all_typed_specs`
Representative construction	`PTA.BlowupB.keevashTypedSpec_of_closed765`
Exact classification	`PTA.BlowupB.FiniteCensus.census_complete`
Number of representatives	`PTA.BlowupB.FiniteCensus.representativeMasks_card`
Catalogue bridge	`PTA.BlowupB.FiniteCensus.representative_closed765`
Theorem D	`PTA.BlowupB.FiniteCensus.keevashTypedSpec_of_admissible`

`KeevashTypedSpec` is a definition, not an axiom attributed to Keevash. Theorem D's conclusion retains the host hypotheses in (2.7). It does not claim a decomposition for every integrally divisible graph. The real inverse retains local profiles as well as global type totals. Theorem C counts ordered disagreements; the manuscript's unordered edit cost is half that count.

`Core`, `Families` and `PaperIApplications` have four, seven and four final queries respectively. These are the exact audited targets; a default `lake build` or single umbrella target was not the validation command. Internal module names are retained to preserve the validated sources.

The table is a source correspondence, not a list of fifteen individual axiom queries. Additional per-module queries appear in the historical logs. In particular, `teoremaB_balanced` has a successful module receipt in `BlowupB/TeoremaBBalanced.lean`, but is absent from both the fifteen final queries and the recorded Paper II phase query list. Its source compiles; this companion does not claim a separate final axiom audit of that declaration.

The name `KeevashTypedSpec` records the intended graph-level interface. `BlowupB/KeevashBridge.lean` defines it and proves conditional consumers; it does not import a formalization of Keevash's labelled-complex Theorem 3.1. `isIntDiv_iff_physDiv_uniform`, in `BlowupB/Uniform.lean`, assumes `IsCompleteBlowup` and `Reps p 5`. Its scope is therefore a complete allowed host with five representatives per class. It cannot establish the corresponding lattice identification for an arbitrary host in (2.7). Section 2.3 of the manuscript records the comparison and this limitation; no equivalence with the general labelled theorem is claimed.

The four-colour query in `PaperIApplications` is now `NashWilliams18.PaperII.margin_four_colour_decomposition`, the bridge to `nearlyComplete_F123` used by Paper I. Its proof dependency audit requires this bridge and excludes the former consumer. The two papers now certify the same four-colour interface, within their separate entry closures.

4. Census and conserved families

The finite proof uses a reversible encoding of twenty triangle types and twenty-six boxes determined by forbidden partners. Coverage certificates provide actual permutations of labels. Minimality proves uniqueness of the representative. `census_complete` combines these facts into an exact classification with 765 representatives.

The construction predicate need not itself be invariant under every permutation. The final proof transports the established typical-host property along the inverse permutation; it does not infer catalogue membership for the original labelling. All 28 bridge modules compiled in the complete closure.

For the conserved families, the proof supplies core and non-core certificates at a common scale before applying the splitting transfer. The physical reserve and catalogue are fixed before the

residue. Weights count all catalogue occurrences of each arc, and the degree bound concerns total directed degree. Python generated candidate certificates; Lean checked coverage, uniqueness and transfer. The census bridge uses no `native_decide` or new mathematical axiom.

5. Reproduction and evidence

For a static check, extract the selected archive and run from its root:

```
python tools/verify_attribution.py
```

This checks all current source hashes, reconstructs the exact historical sources from `attribution/source_changes.json`, checks 1,510 successful historical receipts and 3,096 project-import object-hash links, and verifies the internal manifest. It does not invoke Lean. The 3,029 existing evidence files remain byte-identical to the clean predecessor.

`evidence/verification.json` records the campaign result, including fifteen standard-axiom queries. `evidence/build_plan.json` gives the topological plan. Each module has a receipt in `evidence/receipts/` and compiler output in `evidence/module_logs/`; `evidence/entry_audit.log` collects the three entry outputs. At assembly, source hashes matched the receipts and all compiled objects on disk matched their recorded hashes. Compiled objects are not included in this archive.

The campaign ran in several invocations, resuming only objects with valid receipts from that campaign. The final Paper II invocation reports 517 modules newly compiled and 370 reused from earlier invocations of the same clean campaign, totalling the 887 Paper II-phase modules. These 370 are not an imported project cache from an earlier campaign. All 517 modules compiled in the final invocation used the raised memory limit; this changed resource limits, not mathematical sources. The first 23 modules were recompiled against the final Mathlib checkout of the same revision, with identical output bytes, as recorded in `evidence/early_module_recheck.txt`. These are parts of one local clean campaign, not independent replications.

For a fresh compilation, install `leanprover/lean4:v4.35.0-rc3`, prepare Mathlib at `c55e6e786f49471c72fbddbec5415808896aec1e` and the recorded dependency revisions, and obtain their cache with `lake exe cache get`. Preserve the lock file; do not run `lake update`. Compile the supplied `source/` tree in a separate output directory and generate new receipts. The historical `tools/build.py` and `tools/align.py` use fixed Windows workspace paths and require adaptation; they are campaign records, not a portable one-command reproducer. In particular, the alignment script reconstructs the pre-correction sources and must not overwrite the selected corrected tree. The original campaign instructions remain in `evidence/BUILD_README.md`.

Some source comments point to historical research notes or generators outside the archive. The review inventory records their exact paths, distinguishes explicit `cegis/` references from other note names, and checks that these are not project imports or required runtime reads. They are not supplied as reproducibility resources. The frozen Lean proofs and receipts are preserved without editing those comments. The audit inventory is `02_validation/SOURCE_NOTE_INVENTORY.json` in the series repository.

Relative links and audit-script paths describe the complete series monorepository. Copying only `preprints/PAPER_II/` does not produce a standalone distribution. The source ZIP's static verifier is a separate, self-contained check of its packaged files.

This editorial integration executes no Lean build. Its static audit verifies the supplied evidence and exact source correspondence; it neither inspects omitted compiled objects anew nor claims an independent reproduction.

6. Written proof and formal correspondence

The mathematical appendices of Paper II now integrate the six written-proof dossiers at repository commit `61f752fc87bff6f5654725b23ce10ad4302364ab`. Their source correspondence is listed in Section 7. The authoring operation preserves the theorem statements and frozen sources. It adds exposition; it does not infer mathematical validity from typesetting or finite arithmetic checks.

The correspondence has two levels. The declarations in Section 3 state the principal results. The construction modules in Section 7 supply the real inverse, signed certificates, refiner, semidense geometry, component balancing and conserved catalogue. Auxiliary ordinary-refiner, robust-matching, packing and cover-down results remain explicit dependencies. The finite family tables are also required by the universal classification: example components are not a substitute for those tables.

The semidense theorem, the balanced F1–F3 deduction and Lemma SC are distinct. In the semidense interface, ordered disagreement is twice the manuscript's unordered edit count. In the conserved catalogue, degree means total in-degree plus out-degree and the load sums over all catalogue members. Vertex splitting can change the order of the host, so relative supply and degree budgets are rescaled before projection. These conventions are material to the correspondence.

The manuscript, all mathematical appendices and auxiliary statements and this companion are available in full in English and Spanish. Both editions retain the same mathematical expressions, declaration identifiers and source hashes. The manuscript and this companion constitute their first public edition, preprint 1.0.

The selected archive documents the completed aligned campaign and its subsequent header-only and README-only revisions. Project sources and original packaging materials use Apache-2.0; dependency licences remain intact. The manuscript and this companion use CC BY 4.0. The recorded compilation is one local campaign, not independent reproduction.

7. Map of the mathematical appendices

The paths below are relative to `source/PartitionedTriangleAbsorption/` in the fixed Paper II archive, except where a table gives a more specific base. Appendix A corresponds to the bounded-inverse and terminal components used by the entries in Section 3. Appendix D corresponds to `BlowupB/VertexSplit/{Basic,Div,Decomp,Supply,Transfer,ComponentsT}.lean`. The remaining correspondence is given below. Some tables use braces or wildcards to describe a family of files; these are reading aids, not import commands.

Appendix A

Argument	Frozen modules
Complete allowed-host real inverse	`BlowupB/PaddedGeneral/RealInverse/{Basic,Global,Modes,Factors,Stages}.lean`
Real inverse and consumer interfaces	`BlowupB/PaddedGeneral/RealInverse/{Producer,Uniform,ProducerConsumer,Consumer}.lean`
Bounded signed certificates	`TypedBoundedCoefficients.lean`, `TypedLatticePadding.lean` and their dependencies
Physical terminal gadgets and budgets	`BlowupB/CertAbsorber/{Basic,Cert,Gadget,Decomp,Host,Greedy,Budgets,Terminal,Produce}.lean`

Appendix B

Argument	Sources
Local lattice	`BlowupB/UniversalProducer/PairLattice.lean`
Ordinary finite refiner	`UniversalRefiner/{Basic,Level,Hierarchy,Heap,Assembly,Loads,Main}.lean`; `ParityChain.lean`, `ModThreeLayer.lean`
Virtual library and routing	`BlowupB/UniversalProducer/{VirtualRefiner,LabelWalk,WeightedRouting,WeightedRoutingChoice,WeightedRoutingBudget,RoutedEvenProducer}.lean`
Charge group and finite patterns	`BlowupB/UniversalProducer/{ChargeAtoms,Patterns,PatternSupply,IterLayers}.lean`
Ledger and torsion wraps	`BlowupB/UniversalProducer/{GroupLedger,GroupLayer,Reduction,ReductionArith}.lean`; `BlowupB/StructuralF3General/TokenLedger.lean`
Private absorbers	Appendix A.4–A.6; `BlowupB/UniversalProducer/Refiner.lean`
Quantitative consumer	`BlowupB/PaddedGeneral/{InverseScales,ScaledPipeline}.lean`; `BlowupB/PaddedGeneral/RealInverse/{ProducerConsumer,Consumer}.lean`

Appendix C

Paths in this table are relative to `BlowupB/FODirect/`, except `PhysicalHingeSupply/Template.lean`, which is relative to the project namespace root.

Step	Frozen sources
Host hypotheses and deletion	`Geometry.lean`, `GeometryDeletion.lean`
Short-walk stability	`WalkSupply.lean`, `WalkCounts.lean`, `WalkConsequences.lean`, `HostWalkQueries.lean`
Integral certificate	`CertificateIncidence.lean`, `CertificatePeeling.lean`, `CertificateColumns.lean`, `CertificateAssembly.lean`, `CertificateHost.lean`
Explicit hinges and physical counts	`HingeTemplates.lean`, `HostHinges.lean`, `HostBadHinges.lean`, `HostCrossHinges.lean`, `HingeCounting.lean`, `HostHingeSupply.lean`; `PhysicalHingeSupply/Template.lean`
Terminal constants	`FiniteTerminal.lean`, `LocalProducer.lean`
Real inverse	`RealInternalInverse.lean`, `RealCompleteAlgebra.lean`, `RealCompleteBounds.lean`, `RealCompleteInverse.lean`, `InverseRestriction.lean`, `DiscardedCopies.lean`, `HostRealInverse.lean`
Exact weights	`UniformWeights.lean`, `PositiveWeights.lean`, `WeightProducer.lean`
Universal reserve and closure	`UniversalScales.lean`, `Closure.lean`, `Theorem146.lean`

Appendix E

Paths in this table are relative to `BlowupB/`.

Written step	Source modules
Integer local/scalar criteria	`BalFixer/BalInside.lean`, `BalFixer/TorsInside.lean`, `FinT/Lattice.lean`
PAR component routing	`FinT/BudgetKappa.lean`, `FinT/ChargeU12.lean`, `Flex/Even3.lean`, `Flex/TokenCerts.lean`, `Flex/CompWrappers.lean`
Root tokens and geometric supply	`BalFixer/{RootedTokens,BalSupply,BalSupplyX,BalSupplyY}.lean`
Size and load invariants	`BalFixer/{VertexLayer,VertexLayers,VertexLayersX,VertexLayersY,VertexLayersD}.lean`
Global charge variants	`BalFixer/{BalLibraryChi,BalLibraryChiY,BalLibraryChiYM,ChiTokens,ChiTokensM}.lean`
Torsion chain	`TorsionQ/{Chain,Finite,Refiner,Value}.lean`; `BalFixer/BalLibraryTors.lean`
Congruence potentials	`BalFixer/{Pot3,PotQ,PotQ6}.lean`
Quantitative component closure	`BalFixer/BalComponent*.lean`, `BalFixer/BalScales*.lean`, `BalFixer/CompBal*.lean`, `BalFixer/FlexL.lean`
Family transport	`Flex/ParAssembly.lean`, `Flex/Par/`, `Flex/ParClosed/`, `BalFixer/Fam/`, `BalFixer/All.lean`
Final coverage	`TeoremaB748.lean`, `TeoremaB765.lean`, the census imported by the Paper II family entrypoint

Appendix F

Paths in this table are relative to `BlowupB/LatinCore/`.

Written argument	Frozen sources
Flow rounding and tensor inverse	`RealTensorFilling.lean`, integer-flow and bounded-integer-filling dependencies
Four-permutation graph	`Expander/BinomialEstimates.lean`, `Expander/PermutationExpander.lean`
Profile normalization and rectangles	`DenseProducer/{ExpanderFlow,Faces,Counting,Profile,Filling}.lean`
Positive exact coverage and dense bounds	`Positivization/{ExactCover,Load}.lean`, `DenseProducer/{DenseVc,Main}.lean`
Greedy hub partition and rooted load	`Cycles/{GreedyPartition,DenseReduction,DenseAssemblyBounds,DenseOldLoad,DenseAssembly}.lean`
Eight-permutation root network	`Expander/EightPermutation.lean`, `RootChannels.lean`, `RootPaths.lean`, `RootLift.lean`, `RootInput.lean`
Completed fixed catalogue	`CompletedCatalogue.lean`, `RootMesoscopic.lean`, `RootCatalogue.lean`, `RootCatalogueBudget.lean`

8. Auxiliary statements: exact source locations

Appendix G of the manuscript states the auxiliary results used in the assembly. The following paths are relative to `source/PartitionedTriangleAbsorption/` in the selected Paper II freeze. Names identify declarations within the indicated module, not new public entry queries. The evidence is the source and its recorded dependency closure; this revision adds no Lean build or axiom query.

- **G.1**: `ReturnedBasic.lean`; `AntiEdgeful`.
- **G.2**: `BlowupB/PaddedNibble/TypedEffectiveConsumer.lean`; `typed_nibble_packing_near_effective`.
- **G.2**: `BlowupB/PaddedNibble/TypedNibbleAdapter.lean`; `typed_nibble_packing`.
- **G.2**: `BlowupB/PaddedNibble/Nibble/PolynomialThreshold.lean`; `packingPolynomial`.
- **G.2**: `BlowupB/PaddedQuantitative/EffectiveSchedule.lean`; `loadTolerance`.
- **G.2**: `BlowupB/PaddedNibble/ReserveConstruction.lean`; `exists_reserve_of_decomposition`.
- **G.3**: `BlowupB/PaddedNibble/TypedEffectiveConsumer.lean`; `typed_cover_down_absorption_effective`.
- **B.3/G.4**: `BlowupB/UniversalProducer/RoutedEvenProducer.lean`; `evenRefinerProducer_routed`.
- **G.4/H.5, C.5a**: `TypedRefinerF0/RobustSlots.lean`; `exists_robust`.
- **B.3/G.4/H.6**: `UniversalRefiner/Main.lean`; `exists_universal_refiner`.
- **G.5**: `BlowupB/LatinCore/WeightedPathStep.lean`; `exists_weighted_path`.
- **B.4**: `BlowupB/UniversalProducer/IterLayers.lean`; `iterate_layers`.
- **B.6**: `BlowupB/PaddedF12/HostGap.lean`; `degree_gap_of_guarded`.
- **C.5a**: `F0FlexibleCompressed.lean`; `compatible_producer`.
- **C.5a**: `TypedRefinerF0/ChargeLayer.lean`; `charge_layer`.
- **C.5a**: `CompatibleShared/Host.lean`; `compatible_shared_host`.
- **C.6**: `BlowupB/F0Direct/UniversalScales.lean`; `universal_from_small_reserve`.

- **E.8**: `BlowupB/PaddedF12/PaddedDefinitions.lean`; `KeevashPaddedSpec`.
- **E.8**: `BlowupB/PaddedComponents/ReductionLarge.lean`; `keevashPaddedSpec_of_certsₛ`.
- **E.8**: `BlowupB/KeevashPaddedMain.lean`; `keevashTypedSpec_of_padded`.
- **8**: `BlowupB/BalFixer/FlexL.lean`; `UniversalCertFlexL`.
- **8**: `BlowupB/BalFixer/CompL.lean`; `compScalesL`.
- **8**: `BlowupB/BalFixer/CompL.lean`; `universalCertFlexL_of_split`.

Appendix H now gives written mathematical proofs of the compatible charge library (H.1–H.4), robust slots (H.5), ordinary refiner (H.6), packing (H.7–H.8) and whole-block reserve selection (H.9). H.3 derives the layer factors in B.4. G.3 and G.5 give the cover-down and weighted-path reductions. The finite family certificates remain explicit data in the frozen sources. This exposition preserves the proof bodies of the selected aligned freeze and adds no Lean build or axiom queries. The written reconstruction and finite regression checks are auxiliary development material; the source locations above identify the frozen proof dependencies. Finite checks are transcription evidence, not premises of the universal theorems.

The 120 connected non-bipartite link cases comprise 93 three-step cases and 27 further padded cases. The other routes cover 271 PAR cases and 357 BAL cases, followed by the 17 conserved cases. The padded specification is a proved intermediate interface, not an additional mathematical axiom or a theorem attributed to an unavailable manuscript.

The repository check scripts expect the complete frozen source tree at their documented relative paths. A documents-only sparse checkout is insufficient. Restore the source archive or use a complete checkout before running those scripts; this is distinct from a new Lean reconstruction.

Use of artificial intelligence and computational tools

Claude, by Anthropic, and ChatGPT/Codex, by OpenAI, were used to explore and check arguments and to prepare the manuscript. Aristotle, by Harmonic, contributed to exploring approaches, searching for counterexamples, developing proofs, and formalizing and reviewing them in Lean. Its role was not limited to translating finished proofs.

Certo [3] and Jacobian [4] were used to explore finite instances, check identities and assess candidate constructions. Finite checks were not treated as proofs of universal statements. The census search generated finite certificates. Lean then verified coverage of the admissible universe, uniqueness up to actual permutations, and the connection to the construction theorems; these are stronger assertions than compiling a list of examples.

The author remains responsible for the arguments, citations, code and presentation. No AI system is listed as an author.

References

[1] J. P. Traverso Gianini, *Triangle decompositions II: Robust absorption for typed triangles*, preprint 1.0 (2026). https://doi.org/10.5281/zenodo.23502224. Document.

[2] J. P. Traverso Gianini, *Triangle decompositions I: Formalizing the Nash–Williams theorem in Lean*, preprint 1.0 (2026). https://doi.org/10.5281/zenodo.23428173. Document.

[3] J. P. Traverso Gianini, *Certo*. https://github.com/jtraverso/certo-math.

[4] *Jacobian*. https://github.com/morluto/jacobian.